

データベースとRAGシステムの実装

本日の学習内容

1. データベースの基礎 - なぜデータベースが必要なのか？
2. SQLの基本 - データを操作してみよう
3. 全文検索 - 大量の文書から情報を探す
4. RAGシステム - AIに外部知識を与える技術
5. 実装演習 - 実際にRAGシステムを作る

Part 1: データベースの基礎

なぜデータベースが必要なのか？

データ管理の問題

例：料理レシピを保存したい場合

✕ テキストファイルに保存

料理名：カレーライス、材料：じゃがいも、にんじん...
料理名：肉じゃが、材料：じゃがいも、牛肉...

- 特定のレシピを探すのが大変
- データの更新が難しい
- 複数人で同時に使えない

データベースの利点

✓ データベースを使うと

- **検索が速い:** 「じゃがいもを使う料理」を瞬時に検索
- **データの整合性:** 重複や矛盾を防ぐ
- **同時アクセス:** 複数のユーザーが同時に利用可能
- **安全性:** データのバックアップや復元が容易

データベースの種類

リレーショナルデータベース（表形式）

- MySQL, PostgreSQL, SQLite
- 表（テーブル）としてデータを整理
- **今日はこれを学習します**

NoSQLデータベース（柔軟な形式）

- MongoDB, Redis, Elasticsearch
- ドキュメント、キーバリュー、グラフなど様々な形式

リレーショナルデータベースとは？

Excelのような表でデータを管理

id	name	category	ingredients
1	カレーライス	洋食	じゃがいも、にんじん
2	肉じゃが	和食	じゃがいも、牛肉

- 表 = テーブル (Table)
- 行 = レコード (Record)
- 列 = カラム (Column)

テーブルの基本構造

recipesテーブル

id	name	category	ingredients
1	カレーライス	洋食	じゃがいも、にんじん
2	肉じゃが	和食	じゃがいも、牛肉

- **id**: 主キー（各レコードを一意に識別）
- **name, type, ingredients**: データを保存する列

SQLiteについて学ぼう

SQLite = 軽量で使いやすいデータベース

特徴：

- **ファイルベース**: 1つのファイルがデータベース
- **サーバー不要**: インストールして即使える
- **無料**: 完全に無料で使える
- **軽量**: スマホアプリにも使われている

最初のデータベース学習に最適！

Part 2: SQLの基本

データを操作してみよう

SQL（エスキューエル）とは？

Structured Query Language = 構造化問い合わせ言語

データベースと対話するための言語

-- 人間の言葉：「野菜カテゴリの料理を全部見せて」

-- SQL:

```
SELECT * FROM recipes WHERE category = '野菜';
```

世界中のデータベースで使われる標準言語

SQLの4つの基本操作（CRUD）

1. **Create（作成）** : INSERT - データを追加
2. **Read（読取）** : SELECT - データを取得
3. **Update（更新）** : UPDATE - データを変更
4. **Delete（削除）** : DELETE - データを削除

この4つを覚えれば基本的な操作ができます！

Step 1: データを見る (SELECT)

基本構文

```
SELECT 列名 FROM テーブル名;
```

例1: 全てのデータを取得

```
SELECT * FROM recipes;
```

* は「全ての列」を意味する

例2: 特定の列だけ取得

```
SELECT name, ingredients FROM recipes;
```

料理名と材料だけを表示

SELECT文の実行結果

```
SELECT name, ingredients FROM recipes;
```

結果:

name	ingredients
カレーライス	じゃがいも、にんじん、玉ねぎ
肉じゃが	じゃがいも、牛肉、玉ねぎ
野菜炒め	キャベツ、にんじん、ピーマン

指定した列だけが表示される

Step 2: データを絞り込む (WHERE)

WHERE句 = 条件を指定して絞り込み

```
SELECT name, ingredients  
FROM recipes  
WHERE food_category_1 = '野菜';
```

読み方:

「recipesテーブルから、カテゴリが'野菜'のレシピの名前と材料を取得」

WHERE句の様々な条件

-- 等しい

WHERE category = '和食'

-- 等しくない

WHERE category != '和食'

-- 数値の比較

WHERE cooking_time > 30

-- 複数条件 (AND)

WHERE category = '和食' AND cooking_time < 30

-- 複数条件 (OR)

WHERE category = '和食' OR category = '中華'

-- 部分一致 (LIKE)

WHERE name LIKE '%カレー%'

Step 3: データを並び替える (ORDER BY)

ORDER BY句 = ソート (並び替え)

```
-- 昇順 (小さい→大きい、あいうえお順)  
SELECT name, cooking_time  
FROM recipes  
ORDER BY cooking_time ASC;
```

- **ASC**: 昇順 (Ascending)

```
-- 降順（大きい→小さい、逆順）  
SELECT name, cooking_time  
FROM recipes  
ORDER BY cooking_time DESC;
```

- **DESC:** 降順 (Descending)

ORDER BY の実行例

```
SELECT name, cooking_time
FROM recipes
ORDER BY cooking_time ASC;
```

結果:

name	cooking_time
-----	-----
野菜炒め	10
肉じゃが	30
カレーライス	45

調理時間が短い順に並ぶ

Step 4: データを追加する (INSERT)

INSERT文 = 新しいレコードを追加

```
INSERT INTO recipes (name, abstract, ingredients, steps)
VALUES (
    '新しい料理',
    '料理の概要',
    '材料のリスト',
    '調理手順'
);
```

ポイント:

- 列名と値の順番を合わせる
- 文字列は'シングルクォート'で囲む

INSERT文の実践例

```
INSERT INTO recipes (name, food_category_1, ingredients, steps)
VALUES (
    'トマトパスタ',
    '洋食',
    'パスタ、トマト、にんにく、オリーブオイル',
    '1.パスタを茹でる 2.ソースを作る 3.混ぜる'
);
```

実行後:

✓ 1行追加されました

新しいレシピがテーブルに追加される

Step 5: データを更新する (UPDATE)

UPDATE文 = 既存データの変更

```
UPDATE recipes  
SET abstract = 'パーティー向け料理'  
WHERE name = 'トマトパスタ';
```

⚠ 重要: WHERE句を忘れると全レコードが更新される！

UPDATE文の注意点

--  危険: WHERE句がないと全部更新される

```
UPDATE recipes  
SET category = '洋食';
```

--  安全: WHERE句で対象を限定

```
UPDATE recipes  
SET category = '洋食'  
WHERE name = 'トマトパスタ';
```

必ずWHERE句で対象を指定しよう

Step 6: データを削除する (DELETE)

DELETE文 = レコードの削除

```
DELETE FROM recipes  
WHERE name = 'トマトパスタ';
```

⚠ **超重要:** WHERE句を忘れると全データが削除される！

-- ❌ 絶対NG: 全データが消える

```
DELETE FROM recipes;
```

-- ✅ 正しい: 特定のレコードのみ削除

```
DELETE FROM recipes WHERE id = 10;
```

Part 3: 全文検索

大量の文書から情報を探す

全文検索とは？

通常の検索（LIKE）の問題点

```
-- 遅い！大量のデータには不向き  
SELECT * FROM recipes  
WHERE steps LIKE '%レンジ%';
```

- 全データを順番にスキャンする必要がある
- データ量が増えると極端に遅くなる
- 複雑な検索条件に対応しづらい

n-gramによる全文検索

n-gram = 文字列をn文字ずつ分割する手法

この手法を使って高速な全文検索を実現

1. 事前に文書をn-gramに分割してインデックス化
2. 検索時はクエリもn-gramに分割
3. 共通のn-gramを持つ文書を高速に発見
4. 類似度を計算してランキング

日本語の前処理が重要

問題: 日本語には単語の区切りがない

「電子レンジで温めます」
→ どこで単語が区切れる？

解決策: 形態素解析 vs n-gram

形態素解析

- MeCab、Kuromojiなどを使用
- 辞書が必要
- 精度が高いが複雑

n-gram (今日使用)

- 辞書不要
- シンプルで実装が簡単
- どんな言語でも使える

n-gram解析とは？

n-gram = n文字ずつ分割する手法

例: 「レンジで」を3-gram (トライグラム) で分割

「レンジで」→「レン」「ンジ」「ジで」

利点:

- 辞書不要 (どんな言語でも使える)
- シンプルで実装が簡単
- 未知語にも対応
- 表記ゆれにもある程度対応

n-gramの実例 - 2-gram

2-gram (バイグラム)

「カレー」 → 「カレ」「レー」

「データベース」 → 「デー」「ータ」「タベ」「ベー」「ース」

- 短い単位で分割
- より細かいマッチングが可能
- 誤検出も増える可能性

n-gramの実例 - 3-gram

3-gram (トライグラム)

「カレーライス」 → 「カレー」「レーラ」「ーライ」「ライス」

「電子レンジ」 → 「電子レ」「子レン」「レンジ」

n-gramによる検索の仕組み

1. 文書を登録時
「Pythonプログラミング」
→ ["pyt", "yth", "tho", "hon", ...] をDBに保存
2. 検索時
クエリ「Python学習」
→ ["pyt", "yth", "tho", "hon", ...] に分割
3. 共通のn-gramを持つ文書を検索
「Pythonプログラミング」が候補に
4. 類似度を計算してランキング
Jaccard係数などを使用

Part 4: RAGシステム

AIに外部知識を与える技術

RAG（ラグ）とは？

Retrieval Augmented Generation

= 検索拡張生成

簡単に言うと：

AIが答える前に、関連する情報を検索して参考にする仕組み

なぜRAGが必要？

大規模言語モデル（LLM）の限界

✗ 学習データの知識しか持たない

- 以前の情報しか知らない
- 企業の内部情報は知らない
- 最新のニュースは知らない

✓ RAGで解決

- 外部データベースから最新情報を取得
- 社内文書を参照して回答

RAGの仕組み - 全体像

1. ユーザーの質問
「Pythonで機械学習する方法は？」
↓
2. 関連文書を検索 (Retrieval)
データベースから関連する文書を取得
↓
3. 検索結果とLLMの知識を統合
「以下の文書を参考に回答してください: ...」
↓
4. 回答を生成 (Generation)
「Pythonで機械学習を行うには...」

RAGのメリット

✓ 最新情報の提供

- データベースを更新すれば最新情報に対応

✓ 精度と信頼性の向上

- 根拠となる文書を示せる
- ハルシネーション（嘘）を減らせる

✓ 柔軟性

- 質問応答システム
- チャットボット

RAGの課題

⚠ データ品質への依存

- 検索元のデータが古いと古い情報を提供

⚠ 処理時間

- 検索 + 生成で時間がかかる

⚠ 誤情報のリスク

- データベースに誤った情報があると誤回答

RAGの実装方法は2種類

1. キーワードベース（今日の実装）

- n-gramで文字列の類似度を計算
- シンプルで実装が簡単
- 計算コストが低い

2. ベクトルベース（発展）

- 埋め込みモデルで意味的な類似度を計算
- より高精度
- 計算コストが高い

Part 5: RAGシステムの実装

n-gramベースの文書検索を作ろう

実装の全体構成

```
SimpleDocumentSearch
├── __init__()           # 初期化
├── _initialize_db()     # データベース作成
├── _extract_ngrams()    # n-gram抽出
├── add_document()       # 文書追加
├── _calculate_similarity() # 類似度計算
└── search()            # 検索実行
```

5つのステップで実装します

Step 1: 必要なライブラリをインポート

```
import sqlite3          # データベース操作
import re                # 正規表現（テキスト処理）
from collections import Counter  # 頻度カウント
from typing import List, Tuple, Dict  # 型ヒント
import json              # メタデータの保存
```

全て標準ライブラリ！追加インストール不要

Step 2: クラスの初期化

```
class SimpleDocumentSearch:
    def __init__(self, db_path: str = "rag_database.db", n: int = 3):
        """
        Args:
            db_path: データベースファイルのパス
            n: n-gramのn (デフォルトは3-gram)
        """
        self.db_path = db_path # DBファイル名を保存
        self.n = n # n-gramのサイズ
        self.conn = sqlite3.connect(db_path) # DB接続
        self._initialize_db() # テーブル作成
```

使い方:

```
search_engine = SimpleDocumentSearch(n=3)
```

Step 3: データベースのテーブル作成

```
def _initialize_db(self):  
    """データベースの初期化"""  
    cursor = self.conn.cursor()  
    # 文書を保存するテーブル  
    cursor.execute("""  
        CREATE TABLE IF NOT EXISTS documents (  
            id INTEGER PRIMARY KEY AUTOINCREMENT,  
            content TEXT NOT NULL,  
            metadata TEXT,  
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
        )  
    """)
```

IF NOT EXISTS : 既にテーブルがあればスキップ

テーブル設計の詳細

documentsテーブル

id	content	metadata	created_at
1	Python...	{...}	2025-11-04
2	SQLite...	{...}	2025-11-04

- **id**: 自動採番の主キー
- **content**: 文書の本文
- **metadata**: 追加情報（JSON形式）
- **created_at**: 登録日時（自動）

n-gramテーブルの作成

```
# n-gramを保存するテーブル
cursor.execute("""
    CREATE TABLE IF NOT EXISTS ngrams (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        document_id INTEGER NOT NULL,
        ngram TEXT NOT NULL,
        frequency INTEGER NOT NULL,
        FOREIGN KEY (document_id) REFERENCES documents(id)
    )
""")
# 検索を高速化するインデックス
cursor.execute("""
    CREATE INDEX IF NOT EXISTS idx_ngrams ON ngrams(ngram)
""")
```

ngramsテーブルの役割

文書から抽出したn-gramを保存

id	document_id	ngram	frequency
1	1	pyt	1
2	1	yth	1
3	1	tho	1
4	1	hon	1

インデックスで「ngram」列を高速検索

Step 4: n-gram抽出の実装

```
def _extract_ngrams(self, text: str) -> Counter:
    """テキストからn-gramを抽出"""
    # ステップ1: 正規化（前処理）
    text = text.lower() # 小文字に統一
    text = re.sub(r'[^\\w\\s]', '', text) # 記号を削除
    # ステップ2: n-gramに分割
    ngrams = []
    for i in range(len(text) - self.n + 1):
        ngram = text[i:i + self.n]
        if ngram.strip(): # 空白のみは除外
            ngrams.append(ngram)
    # ステップ3: 頻度をカウント
    return Counter(ngrams)
```

n-gram抽出の動作例

```
text = "Python学習"
```

```
n = 3
```

```
# ステップ1: 正規化
```

```
"Python学習" → "python学習"
```

```
# ステップ2: 3-gramに分割
```

```
"python学習"
```

```
→ ["pyt", "yth", "tho", "hon", "on学", "n学習", "学習"]
```

```
# ステップ3: 頻度カウント
```

```
Counter({'pyt': 1, 'yth': 1, 'tho': 1, ...})
```

Step 5: 文書の追加

```
def add_document(self, content: str, metadata: Dict = None):  
    """文書をデータベースに追加"""  
    cursor = self.conn.cursor()  
  
    # ステップ1: 文書を保存  
    metadata_json = json.dumps(metadata) if metadata else None  
    cursor.execute(  
        "INSERT INTO documents (content, metadata) VALUES (?, ?)",  
        (content, metadata_json)  
    )  
    doc_id = cursor.lastrowid # 追加された文書のID取得
```

?: プレースホルダー (SQLインジェクション対策)

文書追加の続き - n-gram保存

```
# ステップ2: n-gramを抽出
ngrams = self._extract_ngrams(content)

# ステップ3: n-gramをDBに保存
for ngram, freq in ngrams.items():
    cursor.execute(
        "INSERT INTO ngrams (document_id, ngram, frequency) "
        "VALUES (?, ?, ?)",
        (doc_id, ngram, freq)
    )

self.conn.commit() # 変更を確定
print(f"文書ID {doc_id} を追加しました")
```


文書追加の流れ

入力: "Pythonは汎用的な言語です"

↓

1. documentsテーブルに保存
→ ID = 1 が割り当てられる

↓

2. n-gram抽出
→ ["pyt", "yth", "tho", "hon", ...]

↓

3. ngramsテーブルに保存
(1, "pyt", 1)
(1, "yth", 1)
(1, "tho", 1)
...

Step 6: 類似度計算 - Jaccard係数

Jaccard係数 = 共通要素の割合

$A = \{1, 2, 3, 4\}$

$B = \{3, 4, 5, 6\}$

共通部分: $\{3, 4\} \rightarrow 2$ 個

全体: $\{1, 2, 3, 4, 5, 6\} \rightarrow 6$ 個

Jaccard係数 = $2 / 6 = 0.33$

0.0 (全く異なる) $\sim$ 1.0 (完全一致)

Jaccard係数の実装

```
def _calculate_similarity(self, query_ngrams: Counter,
                          doc_ngrams: Counter) -> float:
    """Jaccard係数を使って類似度を計算"""

    # n-gramの集合を作成
    query_set = set(query_ngrams.keys())
    doc_set = set(doc_ngrams.keys())
    # 共通部分と全体を計算
    intersection = len(query_set & doc_set) # 積集合
    union = len(query_set | doc_set) # 和集合
    # Jaccard係数
    return intersection / union if union > 0 else 0.0
```

Jaccard係数の具体例

```
query = "Python学習"  
query_ngrams = {"pyt", "yth", "tho", "hon", "on学", "n学習", "学習"}  
  
doc1 = "Pythonプログラミング"  
doc1_ngrams = {"pyt", "yth", "tho", "hon", "onプ", "..."}  
  
# 共通: {"pyt", "yth", "tho", "hon"} = 4個  
# 全体: 11個  
# 類似度 = 4 / 11 = 0.36
```

共通するn-gramが多いほど類似度が高い

Step 7: 検索の実装 - 候補取得

```
def search(self, query: str, top_k: int = 3):  
    """クエリに類似した文書を検索"""  
  
    # ステップ1: クエリからn-gramを抽出  
    query_ngrams = self._extract_ngrams(query)  
  
    if not query_ngrams:  
        return []  
  
    # ステップ2: n-gramが一致する文書を取得  
    query_ngram_list = list(query_ngrams.keys())  
    placeholders = ','.join(['?'] * len(query_ngram_list))
```

検索の実装 - SQL実行

```
# ステップ3: データベースから候補を取得
cursor.execute(f"""
    SELECT DISTINCT d.id, d.content
    FROM documents d
    JOIN ngrams n ON d.id = n.document_id
    WHERE n.ngram IN ({placeholders})
""", query_ngram_list)

candidates = cursor.fetchall()
```

JOINで文書とn-gramを結合

- クエリのn-gramを含む文書だけを取得

検索の実装 - 類似度計算

```
results = []
for doc_id, content in candidates:
    # 文書のn-gramを取得
    cursor.execute("""
        SELECT ngram, frequency
        FROM ngrams WHERE document_id = ?
    """, (doc_id,))
    doc_ngrams = Counter(dict(cursor.fetchall()))
    # 類似度計算
    similarity = self._calculate_similarity(
        query_ngrams, doc_ngrams
    )
    results.append((doc_id, content, similarity))
```

検索の実装 - ソートと返却

```
# ステップ5: 類似度でソート (高い順)
results.sort(key=lambda x: x[2], reverse=True)

# ステップ6: 上位top_k件を返す
return results[:top_k]
```

返り値:

```
[
    (1, "Pythonは汎用的な言語...", 0.85),
    (3, "機械学習はPythonで...", 0.72),
    (5, "データ分析にPythonを...", 0.68)
]
```


検索処理の全体の流れ

1. クエリ "Python機械学習"
↓
2. n-gram抽出 {"pyt", "yth", "tho", "hon", ...}
↓
3. DB検索: これらのn-gramを含む文書を取得
↓
4. 各候補文書との類似度を計算
文書1: 0.85
文書3: 0.72
文書5: 0.68
↓
5. 類似度順にソート
↓
6. 上位3件を返す

実際の使用例

```
# ステップ1: システムの初期化
search_engine = SimpleDocumentSearch(n=3)

# ステップ2: 文書を追加
documents = [
    "Pythonは汎用的なプログラミング言語です。",
    "SQLiteは軽量なデータベースです。",
    "機械学習は人工知能の一分野です。",
    "Webアプリ開発にはFlaskやDjangoが人気です。",
    "自然言語処理はテキストデータを扱う技術です。"
]

for doc in documents:
    search_engine.add_document(doc)
```

検索の実行

```
# ステップ3: 検索を実行
query = "Pythonでの機械学習について"
results = search_engine.search(query, top_k=3)

# ステップ4: 結果を表示
for i, (doc_id, content, score) in enumerate(results):
    print(f"\n結果 {i+1}")
    print(f"ID: {doc_id}")
    print(f"スコア: {score:.3f}")
    print(f"内容: {content}")
```

検索結果の例

検索クエリ: Pythonでの機械学習について

結果 1

ID: 1

スコア: 0.456

内容: Pythonは汎用的なプログラミング言語です。

結果 2

ID: 3

スコア: 0.234

内容: 機械学習は人工知能の一分野です。

LLMとの統合方法

検索結果をLLMのプロンプトに含める

```
# 検索実行
results = search_engine.search("Pythonの特徴は?", top_k=3)

# プロンプト作成
context = "\n".join([content for _, content, _ in results])
prompt = f"""
以下の文書を参考に質問に教えてください。

【参考文書】
{context}

【質問】
Pythonの特徴は何ですか?
"""
```

RAGシステムの完成形

1. ユーザーの質問
「Pythonの特徴は？」
↓
2. 文書検索 (SimpleDocumentSearch)
関連文書を取得
↓
3. プロンプト作成
「以下の文書を参考に...」
↓
4. LLMで回答生成
「Pythonは汎用的で...」
↓
5. ユーザーに返答

システムの統計情報を確認

```
def get_stats(self) -> Dict:
    """データベースの統計情報を取得"""
    cursor = self.conn.cursor()

    cursor.execute("SELECT COUNT(*) FROM documents")
    doc_count = cursor.fetchone()[0]

    cursor.execute("SELECT COUNT(DISTINCT ngram) FROM ngrams")
    unique_ngrams = cursor.fetchone()[0]

    return {
        "total_documents": doc_count,
        "unique_ngrams": unique_ngrams
    }
```

実行例とまとめ

```
# 統計情報の表示
stats = search_engine.get_stats()
print(f"登録文書数: {stats['total_documents']}")
print(f"ユニークn-gram数: {stats['unique_ngrams']}")

# 終了時はDB接続を閉じる
search_engine.close()
```

出力例:

```
登録文書数: 5
ユニークn-gram数: 247
```


n-gramベース vs ベクトルベース

n-gramベース（今回の実装）

- ✓ シンプルで理解しやすい
- ✓ 実装が簡単
- ✓ 計算コストが低い
- ✗ 表記ゆれに弱い（「データベース」「DB」は別物）
- ✗ 意味的な類似性は捉えられない

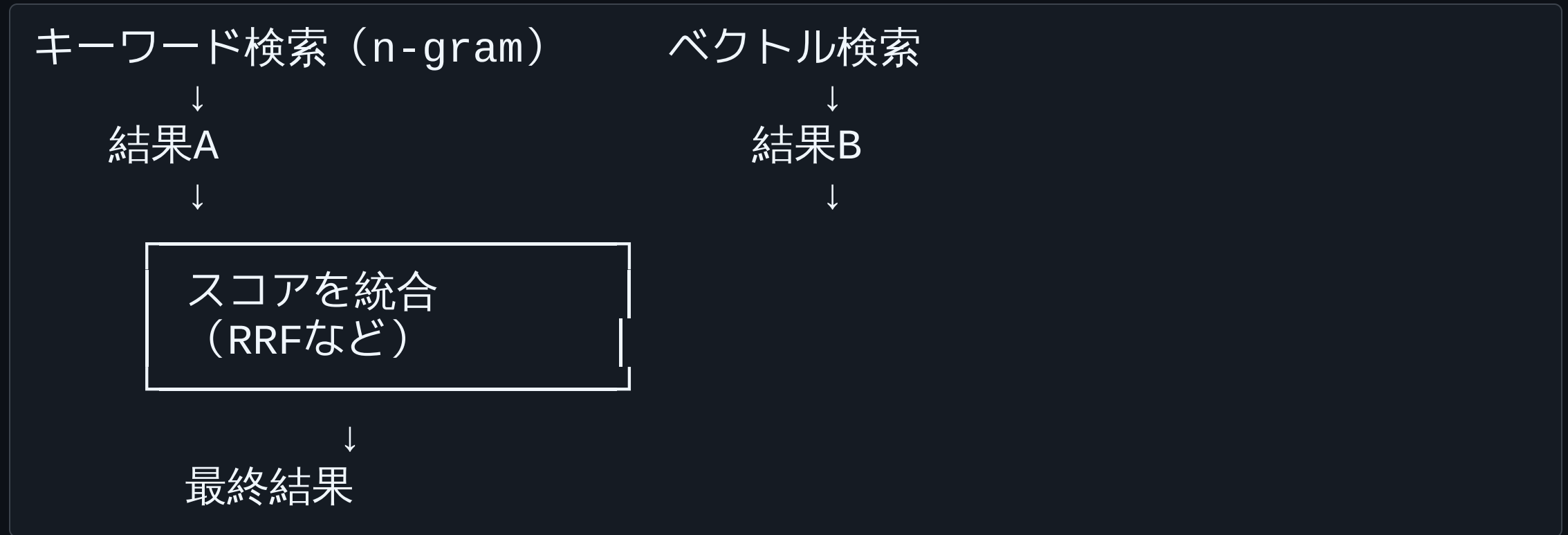
n-gramベース vs ベクトルベース

ベクトルベース（発展）

- ✓ セマンティック（意味的）検索
- ✓ 表記ゆれに強い
- ✓ 多言語対応
- ✗ 埋め込みモデルが必要
- ✗ 計算コストが高い

ハイブリッド検索への発展

両方のいいところ取り



精度と速度のバランスが取れる

実装のポイント総まとめ

1. データベース設計: documentsとngramsの2テーブル
2. インデックス: 検索を高速化する
3. n-gram: 3-gramで文字列を分割
4. Jaccard係数: シンプルな類似度計算
5. 2段階検索: 候補取得→類似度計算
6. LLM統合: 検索結果をプロンプトに含める

よくある質問 Q&A

Q2: 大量の文書でも使える？

A: 数千件程度なら問題ない。それ以上ならベクトル検索やElasticsearchを検討。

Q3: 他の言語でも使える？

A: n-gramなのでどんな言語でも基本的に使える！

本日のまとめ

- ✓ データベースの基礎: テーブル、主キー、外部キー
- ✓ SQLの基本: SELECT, INSERT, UPDATE, DELETE
- ✓ 全文検索: n-gramによる文字列分割
- ✓ RAGの仕組み: 検索→生成の2ステップ
- ✓ 実装: n-gramベースの文書検索システム